

## Лабораторная работа №31-32. Введение в SQLite и Entity Framework Core

### Цель работы:

- Понять, что такое база данных и зачем она нужна в веб-приложении;
- Познакомиться с SQLite — лёгкой встраиваемой базой данных;
- Подключить Entity Framework Core к проекту ASP.NET Core;
- Создать первую миграцию и таблицу через Code First подход;
- Научиться выполнять базовые операции с БД через C# (LINQ);
- Сравнить SQL-запросы и LINQ — увидеть знакомую логику в новом контексте.

### Краткая теория (вводная часть)

#### Зачем нужна база данных?

В предыдущих лабораторных работах данные хранились в обычном списке в памяти. Это работало, но у такого подхода есть критический недостаток: **при перезапуске сервера все данные пропадают**.

#### Сравним подходы хранения данных:

| Способ хранения                            | Где живут данные         | Что происходит при перезапуске         |
|--------------------------------------------|--------------------------|----------------------------------------|
| <code>static List&lt;T&gt;</code> в памяти | RAM (оперативная память) | Всё теряется                           |
| Файл (JSON, txt)                           | Диск                     | Сохраняется, но неудобно работать      |
| База данных                                | Диск (с умным доступом)  | Сохраняется + быстрый поиск/фильтрация |

#### Что такое SQLite?

**SQLite** — это встраиваемая реляционная база данных. В отличие от PostgreSQL или MySQL, SQLite не требует отдельного сервера — вся база хранится в **одном файле** с расширением **.db** прямо в папке вашего проекта.

Почему SQLite идеально подходит для обучения:

| Характеристика   | SQLite                                   |
|------------------|------------------------------------------|
| Установка        | Не нужна — это просто файл               |
| Сервер           | Не нужен                                 |
| Язык запросов    | Стандартный SQL                          |
| Подходит для     | Разработки, обучения, небольших проектов |
| Файл базы данных | имя.db в папке проекта                   |

**Сравнение с C#:** SQLite для базы данных — как `.txt` файл для хранения текста, только намного умнее. Вы работаете с одним файлом, но внутри него — полноценная база с таблицами, индексами и транзакциями.

## Что такое Entity Framework Core?

**Entity Framework Core (EF Core)** — это **ORM (Object-Relational Mapper)** от Microsoft. Он позволяет работать с базой данных через объекты C#, не написывая SQL-запросы вручную.

Как это работает:

**Класс C#**  $\longleftrightarrow$  **Таблица в БД**

| <b>TaskItem</b>             | <b>tasks</b>                                        |
|-----------------------------|-----------------------------------------------------|
| — <b>int Id</b>             | $\longleftrightarrow$ <b>id INTEGER PRIMARY KEY</b> |
| — <b>string Title</b>       | $\longleftrightarrow$ <b>title TEXT NOT NULL</b>    |
| — <b>bool IsCompleted</b>   | $\longleftrightarrow$ <b>is_completed INTEGER</b>   |
| — <b>DateTime CreatedAt</b> | $\longleftrightarrow$ <b>created_at TEXT</b>        |

**EF Core** переводит операции C# в SQL-запросы автоматически:

| C# (LINQ)                    | SQL (генерирует EF Core)                |
|------------------------------|-----------------------------------------|
| <b>db.Tasks.ToList()</b>     | <b>SELECT * FROM tasks</b>              |
| <b>db.Tasks.Find(5)</b>      | <b>SELECT * FROM tasks WHERE id = 5</b> |
| <b>db.Tasks.Add(task)</b>    | <b>INSERT INTO tasks (...)</b>          |
| <b>db.Tasks.Remove(task)</b> | <b>DELETE FROM tasks WHERE id = ...</b> |
| <b>db.SaveChanges()</b>      | <b>Фиксирует все изменения в БД</b>     |

Вы описываете структуру данных через классы C# и работаете с объектами, а не с SQL-строками.

## Подход Code First

В **EF Core** есть несколько подходов к работе с базой данных. Мы будем использовать **Code First**:

1. Вы пишете классы C# (модели)
2. **EF Core автоматически создаёт** структуру таблиц в БД по этим классам
3. При изменении классов создаётся **миграция** — файл с инструкциями по обновлению БД

**Код C#**  $\rightarrow$  **Миграция**  $\rightarrow$  **База данных**

Альтернатива — **Database First**: сначала создаётся БД, потом генерируются классы. Но Code First удобнее при разработке с нуля.

### Базовые SQL-команды (для понимания того, что делает EF Core)

Прежде чем автоматизировать SQL через EF Core, полезно понять, что именно он генерирует:

```
-- Создание таблицы
CREATE TABLE tasks (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    title         TEXT      NOT NULL,
    description    TEXT,
    is_completed  INTEGER NOT NULL DEFAULT 0,
    created_at    TEXT      NOT NULL,
    priority      TEXT      NOT NULL DEFAULT 'Normal'
);
-- Выборка всех записей
SELECT * FROM tasks;
-- Выборка с условием
SELECT * FROM tasks WHERE is_completed = 0;
-- Вставка записи
INSERT INTO tasks (title, description, priority,
is_completed, created_at)
VALUES ('Изучить EF Core', 'Миграции, LINQ', 'High', 0,
'2026-04-04');
-- Обновление записи
UPDATE tasks SET is_completed = 1 WHERE id = 1;
-- Удаление записи
DELETE FROM tasks WHERE id = 3;
```

## Шаг 1. Создание проекта

### 1.1. Создание рабочей директории

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

```
mkdir Lab31-32_EFCore
```

```
cd Lab31-32_EFCore
```

```
mkdir img
```

## 1.2. Создание проекта Web API

1. На этот раз используем шаблон **webapi** — он создаёт проект с поддержкой контроллеров:

```
dotnet new webapi -n TaskDb
```

2. Откройте проект в VS Code:

```
code .
```

3. Откройте терминал и выполните:

```
cd TaskDb
```

```
dotnet restore
```

## 1.3. Добавление .editorconfig

1. В корне папки **Lab31-32\_EFCore** создайте файл **.editorconfig**

2. Добавьте в него следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

## 1.4. Инициализация Git

1. В терминале выполните:

```
cd ..
```

```
git init -b main
```

```
cd TaskDb
```

```
dotnet new gitignore
```

2. Продолжаем работу в терминале:

```
cd ..
```

```
touch README.md
```

```
git add .
```

```
git commit -m "Initial commit: TaskDb Web API project"
```

3. Создайте пустой **public**-репозиторий на GitHub: **Lab31-32\_EFCore**

4. Привяжите и отправьте:

```
git remote add origin <URL_репозитория>
```

```
git push -u origin main
```

5. Сделайте **скриншот терминала** с результатом **push** и сохраните в **img/gitPushLab31-32\_FIO.png**

## Шаг 2. Установка пакетов EF Core

### 2.1. Что такое NuGet?

**NuGet** — менеджер пакетов для .NET.

### 2.2. Установка необходимых пакетов

Для работы с SQLite через EF Core нам нужны три пакета. Убедитесь что вы находитесь в папке TaskDb, и выполните:

```
dotnet add package Microsoft.EntityFrameworkCore --version 10.0.3
```

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 10.0.3
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 10.0.3
```

```
dotnet add package Swashbuckle.AspNetCore
```

Что делает каждый пакет:

| Пакет                             | Назначение                                          |
|-----------------------------------|-----------------------------------------------------|
| <b>EntityFrameworkCore</b>        | Ядро ORM: LINQ, контекст, миграции                  |
| <b>EntityFrameworkCore.Sqlite</b> | Провайдер для SQLite (умеет работать с .db файлами) |
| <b>EntityFrameworkCore.Tools</b>  | Инструменты командной строки: dotnet ef команды     |

### 2.3. Установка инструмента dotnet-ef

**dotnet ef** — консольный инструмент для создания миграций.

Проверьте установку:

```
cd TaskBoardApi
```

```
dotnet ef --version
```

Вы должны увидеть версию:

```
└─>$ dotnet ef --version
Entity Framework Core .NET Command-line Tools
10.0.3
```

Если у вас не установлен, или версия отличающаяся от 10.0.3, то нужно удалить предыдущую и поставить 10.0.3:

```
dotnet tool uninstall --global dotnet-ef
```

```
dotnet tool install --global dotnet-ef --version 10.0.3
```

Далее выполните в терминале:

```
dotnet restore
```

```
dotnet build
```

Убедитесь, что сборка прошла без ошибок.

Откройте файл **TaskDb.csproj** — в нём должны появиться новые записи о пакетах:

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.3" />
  <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.3" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.3" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.3">
    <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    <PrivateAssets>all</PrivateAssets>
  </PackageReference>
  <PackageReference Include="Swashbuckle.AspNetCore" Version="10.1.4" />
</ItemGroup>
```

**ВНИМАНИЕ!** У вас должны совпадать версии пакетов! В данном примере везде версия 10.0.3! Обратите на это внимание, иногда они отличались, в этом случае установите везде одинаковую версию и заново соберите проект (dotnet restore)

Выполните в терминале:

**cd ..**

**git add .**

**git commit -m "Step 2: EF Core SQLite packages installed"**

**cd TaskDb**

### Шаг 3. Создание модели и DbContext

#### 3.1. Создание структуры папок

1. Создайте папки в **TaskDb**:

**mkdir Models Data Controllers**

#### 3.2. Создание модели TaskItem

На этом шаге мы создаём модель данных, которая будет описывать структуру задачи в нашем приложении.

Модель используется Entity Framework Core для создания таблицы в базе данных, а также применяется при передаче данных между API и клиентом.

1. Создайте файл **Models/TaskItem.cs**

2. Подключаем пространство имён

```
using System.ComponentModel.DataAnnotations;
namespace TaskDb.Models;
```

*Эти пространства имён необходимы для работы с атрибутами валидации.*

*С их помощью можно задавать правила для полей модели (например, обязательность заполнения или максимальную длину строки). Эти правила будут*

применяться как при работе API, так и при создании структуры таблицы в базе данных.

### 3. Создаём класс TaskItem

```
public class TaskItem { }
```

Класс TaskItem представляет **одну запись задачи** в системе. Каждый объект этого класса будет соответствовать одной строке в таблице базы данных. Внутри класса мы определим свойства, которые будут хранить информацию о задаче.

**Далее пишем внутри класса**

### 4. Создаём основные поля

```
public int Id { get; set; }  
public bool IsCompleted { get; set; } = false;  
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

- **Id** — уникальный идентификатор задачи. Entity Framework автоматически сделает его первичным ключом.
- **IsCompleted** — флаг выполнения задачи (true / false).
- **CreatedAt** — дата и время создания задачи. Значение по умолчанию устанавливается в текущее время UTC.

*Использование UTC-времени является хорошей практикой для серверных приложений, так как позволяет корректно работать с различными часовыми поясами.*

### 5. Добавляем название задачи

```
[Required]  
[MaxLength(200)]  
public string Title { get; set; } = string.Empty;
```

Поле **Title** хранит название задачи.

**Атрибуты:**

- **[Required]** — поле обязательно для заполнения.
- **[MaxLength(200)]** — максимальная длина строки 200 символов.

### 6. Добавляем описание задачи

```
[MaxLength(1000)]  
public string Description { get; set; } = string.Empty;
```

Поле **Description** содержит подробное описание задачи. Максимальная длина строки ограничена **1000** символами.

## 7. Добавляем приоритет задачи

```
[MaxLength(20)]  
public string Priority { get; set; } = "Normal";
```

Поле **Priority** хранит уровень приоритета задачи (например: Low, Normal, High). По умолчанию устанавливается значение **Normal**.

Разберём атрибуты валидации:

| Атрибут          | Что делает                                                         |
|------------------|--------------------------------------------------------------------|
| [Required]       | Поле обязательно — EF Core создаст NOT NULL в БД                   |
| [MaxLength(200)] | Максимальная длина — EF Core добавит ограничение                   |
| DateTime.UtcNow  | Время в UTC (универсальное) — хорошая практика для серверного кода |

Важно: атрибуты из `System.ComponentModel.DataAnnotations` работают сразу на двух уровнях:

- **Валидация входящих данных** в контроллере API.
- **Формирование структуры таблицы** в базе данных через Entity Framework Core.

Это позволяет описывать правила для данных в одном месте, что упрощает поддержку и уменьшает вероятность ошибок.

## 3.3. Создание DbContext

**DbContext** — центральный класс **Entity Framework Core**, который отвечает за работу с базой данных. Он представляет **сеанс взаимодействия приложения с БД**: через него выполняются запросы, добавление, изменение и удаление данных.

Кроме того, **DbContext** содержит наборы данных **DbSet<T>**, которые представляют таблицы базы данных в виде коллекций объектов.

Создайте файл **Data/AppDbContext.cs** и вручную перепишите следующий код.

### 1. Подключаем пространства имён

```
using Microsoft.EntityFrameworkCore;  
using TaskDb.Models;
```



```
namespace TaskDb.Data;
```

Назначение:

- **Microsoft.EntityFrameworkCore** — содержит основные классы **Entity Framework Core**, включая **DbContext**, **DbSet**, **ModelBuilder**.
- **TaskDb.Models** — подключает наши модели, чтобы мы могли использовать класс **TaskItem**.

## 2. Создаём класс **AppDbContext**

```
public class AppDbContext : DbContext { }
```

Класс **AppDbContext** наследуется от **DbContext**.

Он будет использоваться приложением для:

- подключения к базе данных,
- выполнения запросов,
- отслеживания изменений объектов,
- сохранения данных.

В приложении обычно создаётся **один основной DbContext**, который управляет всеми таблицами.

## 3. Добавляем конструктор

```
public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
```

Конструктор принимает объект **DbContextOptions**, который содержит:

- строку подключения к базе данных,
- используемый провайдер (например SQLite),
- дополнительные настройки EF Core.

Эти параметры передаются базовому классу **DbContext**.

## 4. Добавляем таблицу через **DbSet**

```
public DbSet<TaskItem> Tasks { get; set; }
```

**DbSet<TaskItem>** представляет **таблицу задач** в базе данных.

Через эту коллекцию выполняются основные операции:

- получение данных (`context.Tasks.ToList()`)
- добавление (`context.Tasks.Add(...)`)

- удаление

Каждый объект TaskItem соответствует **одной строке** таблицы.

## 5. Переопределяем метод OnModelCreating

```
protected override void OnModelCreating(ModelBuilder
modelBuilder) {
    base.OnModelCreating(modelBuilder);
}
```

Метод OnModelCreating используется для **дополнительной настройки модели базы данных**.

Через объект ModelBuilder можно:

- добавлять начальные данные.

Этот метод вызывается при создании модели EF Core.

Далее мы пишем внутри метода `OnModelCreating` (после вызова `base`)!!

## 6. Добавляем начальные данные (Seed Data)

```
modelBuilder.Entity<TaskItem>().HasData(
```

Метод `HasData()` используется для задания **начальных данных (seed data)**. Эти записи автоматически добавятся в таблицу при создании базы данных или при выполнении миграций.

### Пример первой записи:

```
modelBuilder.Entity<TaskItem>().HasData(
    new TaskItem {
        Id = 1,
        Title = "Изучить ASP.NET Core",
        Description = "Контроллеры, маршруты, middleware",
        Priority = "High",
        IsCompleted = true,
        CreatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)
    },

```

Важно: при использовании HasData() **идентификаторы (Id) должны быть указаны вручную**, поскольку EF Core должен знать, какие записи уже существуют.

Следующие **две** записи (добавляем после запятой):

```

new TaskItem {
    Id = 2,
    Title = "Подключить SQLite через EF Core",
    Description = "Миграции, DbContext, LINQ-запросы",
    Priority = "High",
    IsCompleted = false,
    CreatedAt = new DateTime(2026, 1, 2, 0, 0, 0, DateTimeKind.Utc)
},
new TaskItem {
    Id = 3,
    Title = "Написать README",
    Description = "Описать структуру проекта",
    Priority = "Normal",
    IsCompleted = false,
    CreatedAt = new DateTime(2026, 1, 3, 0, 0, 0, DateTimeKind.Utc)
}

```

Теперь при создании базы данных таблица `Tasks` будет автоматически заполнена тремя задачами.

### Разберём структуру:

| Элемент                            | Описание                                                               |
|------------------------------------|------------------------------------------------------------------------|
| <b>DbContext</b>                   | Базовый класс EF Core — управляет подключением и операциями с БД       |
| <b>DbContextOptions</b>            | Настройки: строка подключения, провайдер (SQLite/PostgreSQL/...)       |
| <b>DbSet&lt;TaskItem&gt; Tasks</b> | Коллекция задач — отображается на таблицу <code>Tasks</code> в БД      |
| <b>OnModelCreating</b>             | Метод для тонкой настройки схемы БД и начальных данных                 |
| <b>HasData(...)</b>                | <b>Seed data</b> — начальные данные, которые добавятся при создании БД |

После создания **AppDbContext Entity Framework Core** будет знать:

- какие модели являются таблицами,
- какие данные должны присутствовать при создании базы,
- как взаимодействовать с таблицами через код C#.

### 3.4. Регистрация DbContext в Program.cs

На этом шаге необходимо подключить `DbContext` к приложению и настроить работу с базой данных через `Entity Framework Core`.

Откройте файл **Program.cs** и замените его содержимое следующим кодом (удалите ненужный код, оставив только нужные строчки):

```
// Подключает классы Entity Framework Core (DbContext, провайдеры БД)
using Microsoft.EntityFrameworkCore; +
// Подключает наш класс AppDbContext
using TaskDb.Data; +

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Регистрация DbContext с провайдером SQLite
// "TaskDb" — имя ключа в appsettings.json
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("TaskDb"))); +

// CORS — разрешаем запросы от любого источника (для работы с фронтом)
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => +
        policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
});

var app = builder.Build();

if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowAll"); // Применяем созданную CORS-политику +
app.UseAuthorization();
app.MapControllers();

app.Run();
```

## Подробный разбор

### Подключаем необходимые пространства имён

Эти строки необходимы для того, чтобы приложение могло использовать **EF Core** и наш созданный **DbContext**.

### Основная часть настройки приложения

Этот код уже использовался в предыдущих лабораторных работах:

- **AddControllers()** — подключает поддержку **API-контроллеров**
- **AddEndpointsApiExplorer()** — позволяет автоматически находить API-маршруты
- **AddSwaggerGen()** — подключает генерацию документации Swagger

## Регистрация DbContext

Здесь происходит регистрация **AppDbContext** в контейнере зависимостей (**Dependency Injection**).

Что делает этот код:

- **AddDbContext<AppDbContext>()** — сообщает приложению, что нужно использовать наш **DbContext**
- **options.UseSqlite(...)** — указывает, что база данных будет работать через **SQLite**
- **GetConnectionString("TaskDb")** — получает строку подключения из файла **appsettings.json**

После этой регистрации **AppDbContext** можно использовать в контроллерах через внедрение зависимостей.

## Настройка CORS

Этот код добавляет политику **CORS**.

**CORS** (Cross-Origin Resource Sharing) управляет тем, **какие внешние приложения могут отправлять запросы к нашему API**.

В данной конфигурации:

- **AllowAnyOrigin()** — разрешён любой источник
- **AllowAnyMethod()** — разрешены любые HTTP-методы (GET, POST, PUT, DELETE)
- **AllowAnyHeader()** — разрешены любые заголовки

Это удобно при разработке, когда **фронтенд и API работают на разных портах**.

## Создание приложения

Этот метод создаёт объект приложения на основе всех настроек, которые были добавлены в **builder**.

## Подключение Swagger в режиме разработки

Если приложение запущено в режиме разработки, включается **интерфейс Swagger**, позволяющий тестировать API через браузер.

## Middleware приложения

Новая строка здесь:

- **app.UseCors("AllowAll")** — включает ранее созданную CORS-политику для всех запросов к API.

## Запуск приложения

Метод запускает веб-сервер и начинает обработку HTTP-запросов.

После выполнения этого шага:

- приложение знает, как подключаться к базе данных SQLite
- ApplicationDbContext зарегистрирован в системе зависимостей
- контроллеры смогут работать с базой данных
- разрешены запросы от внешних клиентов (например, фронтенд-приложения).

## 3.5. Строка подключения в appsettings.json

Откройте файл **appsettings.json** и добавьте секцию **ConnectionStrings**:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "TaskDb": "Data Source=taskdb.db"
  }
}
```

**Source=taskdb.db** означает, что файл базы данных будет создан в той же папке, где запускается приложение, с именем **taskdb.db**.

### Что такое строка подключения?

Это адрес базы данных. Для **SQLite** — просто путь к файлу. Для PostgreSQL это было бы что-то вроде `Host=localhost;Database=mydb;Username=user;Password=pass`. Хранить её в **appsettings.json** — хорошая практика: можно менять БД без изменения кода.

## Практическое задание

### dotnet build

Убедитесь, что сборка прошла без ошибок. Если есть ошибки — проверьте все `using` директивы и имена `namespace`.

**Выполните в терминале:**

**cd ..**

**git add .**

**git commit -m "Step 3: AppDbContext and TaskItem model created"**

**cd TaskDb**

## Шаг 4. Миграции — создание структуры базы данных

### 4.1. Что такое миграция?

**Миграция** — это файл с инструкциями по изменению структуры базы данных. Когда вы меняете классы C# (модели), нужно обновить и таблицы в БД — для этого создаётся новая миграция.

Класс C# изменился → Создать миграцию → Применить миграцию → БД обновлена

Миграции хранятся в виде файлов в папке Migrations/ и фиксируются в Git. Это позволяет команде разработчиков синхронно обновлять базу данных.

**Аналог из реального мира:** представьте Excel-таблицу, которую заполняет вся команда. Когда нужно добавить новый столбец — вы не просто добавляете его у себя, а фиксируете это изменение в общем журнале. Миграция — это такой журнал изменений структуры БД.

### 4.2. Создание первой миграции

Убедитесь, что находитесь в папке **TaskDb**, и выполните в терминале:

**dotnet ef migrations add InitialCreate**

После выполнения появится папка Migrations/ с тремя файлами:

**Migrations/**

|                                            |                                    |
|--------------------------------------------|------------------------------------|
| — 20260101120000_InitialCreate.cs          | ← сама миграция (Up и Down методы) |
| — 20260101120000_InitialCreate.Designer.cs | ← метаданные для EF Core           |
| — AppDbContextModelSnapshot.cs             | ← текущее состояние схемы          |

Откройте основной файл миграции (**\_InitialCreate.cs**) и изучите его.

### Краткий разбор файла миграции

Он содержит инструкции для создания или изменения структуры базы данных. **Entity Framework Core** генерирует этот код на основе моделей (**TaskItem**) и настроек **DbContext**. Основная задача миграции — **синхронизировать структуру базы данных с C# моделями**.

## Структура файла миграции

Внутри файла находится класс:

```
public partial class InitialCreate : Migration {
```

Этот класс наследуется от **Migration** и содержит два основных метода:

- **Up()** - Описывает изменения, которые нужно применить к базе данных
- **Down()** - Описывает, как отменить эти изменения

## Метод Up()

```
protected override void Up(MigrationBuilder migrationBuilder) {
```

Метод **Up()** выполняется при применении миграции (dotnet ef database update).

В нашем случае он делает два действия:

1. Создает таблицу **Tasks**
2. Добавляет начальные данные

Создание таблицы:

```
migrationBuilder.CreateTable(
```

**Entity Framework** создает таблицу на основе модели **TaskItem**.

Описание колонок:

```
Id = table.Column<int>(type: "INTEGER", nullable: false)
```

Каждое свойство модели превращается в колонку таблицы.

Примеры соответствия:

| Свойство модели | Колонка БД            |
|-----------------|-----------------------|
| Id              | INTEGER PRIMARY KEY   |
| IsCompleted     | INTEGER               |
| CreatedAt       | TEXT                  |
| Title           | TEXT (maxLength 200)  |
| Description     | TEXT (maxLength 1000) |
| Priority        | TEXT (maxLength 20)   |

Первичный ключ задаётся здесь:

```
table.PrimaryKey("PK_Tasks", x => x.Id);
```

## Добавление начальных данных

Далее идёт вставка данных:

```
migrationBuilder.InsertData(
```



Этот код был сгенерирован на основе метода **HasData()** в **AppDbContext**.

В таблицу **Tasks** автоматически добавятся три записи:

- **Изучить ASP.NET Core**
- **Подключить SQLite через EF Core**
- **Написать README**

Эти данные появятся **сразу после создания базы данных**.

### Метод **Down()**

```
protected override void Down(MigrationBuilder migrationBuilder) {
```

Этот метод выполняется при **откате миграции**.

В нашем случае он удаляет таблицу:

```
migrationBuilder.DropTable(  
    name: "Tasks");
```

То есть база данных возвращается в состояние **до применения миграции**.

### Итог

Файл миграции автоматически описывает:

1. **Какие таблицы нужно создать**
2. **Какие колонки они содержат**
3. **Какие ограничения применяются**
4. **Какие начальные данные добавить**

Таким образом, миграции позволяют **управлять изменениями структуры базы данных через код и систему контроля версий (Git)**.

### 4.3. Применение миграции к базе данных

Выполните в терминале: **dotnet ef database update**

После выполнения в папке **TaskDb** появится файл **taskdb.db** — это и есть ваша база данных SQLite.

 **taskdb.db**

Посмотрим, что произошло «под капотом», введите в **терминале**:

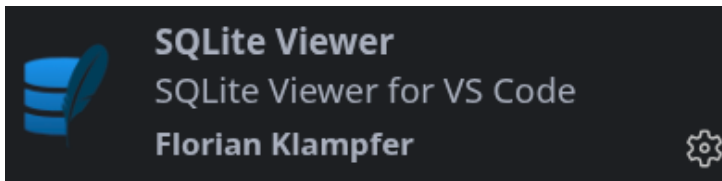
**dotnet ef database update --verbose**

Флаг **--verbose** покажет подробный лог, включая SQL-запросы, которые выполнил EF Core.

## 4.4. Просмотр базы данных через VS Code

Для просмотра содержимого **.db** файла установите расширение:

1. Откройте **Extensions (Ctrl+Shift+X)**
2. Найдите **SQLite Viewer** (от Florian Klampfer)



3. Установите расширение

После установки:

1. В проводнике VS Code найдите файл **taskdb.db**
2. Кликните на него правой кнопкой → **Open With** → **SQLite Viewer**
3. Вы увидите таблицу **Tasks** с тремя начальными записями

| TABLES                 | IsCompleted | CreatedAt           | Title                     | Description                       | Priority |
|------------------------|-------------|---------------------|---------------------------|-----------------------------------|----------|
| > _EFMigrationsHistory | 1           | 2026-01-01 00:00:00 | Изучить ASP.NET Core      | Контроллеры, маршруты, middleware | High     |
| > _EFMigrationsLock    | 2           | 2026-01-02 00:00:00 | Подключить SQLite чере... | Миграции, DbContext, LINQ-запросы | High     |
| > sqlite_sequence      | 3           | 2026-01-03 00:00:00 | Написать README           | Описать структуру проекта         | Normal   |
| > Tasks                | 4           |                     |                           |                                   |          |

## 4.5. Полезные команды dotnet ef

Создать новую миграцию

**dotnet ef migrations add НазваниеМиграции**

Применить все непримененные миграции

**dotnet ef database update**

Откатить последнюю миграцию

**dotnet ef database update ПредыдущаяМиграция**

Посмотреть список миграций и их статус

**dotnet ef migrations list**

Удалить последнюю миграцию (только если она ещё не применена)

**dotnet ef migrations remove**

Посмотреть SQL без применения к БД

**dotnet ef migrations script**

## Практическое задание

1. Выполните **dotnet ef migrations list** и убедитесь, что нету ошибок.
2. Откройте **taskdb.db** через **SQLite Viewer** и сделайте скриншот с таблицей **Tasks** и начальными данными
3. Сохраните скриншот в папку **img** с именем: **step4\_migrationLab31-32\_FIO.png**

Выполните в терминале:

```
cd ..
```

```
git add .
```

```
git commit -m "Step 4: InitialCreate migration applied, taskdb.db created"
```

```
cd TaskDb
```

## Шаг 5. Контроллер с реальной базой данных

### 5.1. Создание контроллера

На этом шаге мы создадим **контроллер API**, который будет работать не с памятью приложения, а с **реальной базой данных** через **Entity Framework Core**.

Контроллер будет обращаться к базе через **AppDbContext**, который мы зарегистрировали ранее в **Program.cs**.

Создайте файл **Controllers/TasksController.cs** и вручную перепишите следующий код.

### Подключаем пространства имён

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using TaskDb.Data;  
using TaskDb.Models;  
namespace TaskDb.Controllers;
```

Новые строки здесь:

- **Microsoft.EntityFrameworkCore** — понадобится для работы с базой данных и выполнения запросов.
- **TaskDb.Data** — подключает наш **AppDbContext**.

## Атрибуты контроллера

```
[ApiController]
[Route("api/[controller]")]
```

### Назначение:

- `[ApiController]` — включает поведение API-контроллера (автоматическая валидация моделей, корректные ответы HTTP).
- `[Route("api/[controller]")]` — задаёт базовый маршрут.

## Объявление класса контроллера

```
[ApiController]
[Route("api/[controller]")]
public class TasksController : ControllerBase { }+
```

**ControllerBase** — базовый класс для **API-контроллеров**.

Он содержит методы для формирования HTTP-ответов (`Ok()`, `NotFound()`, `BadRequest()` и др.).

### Далее пишем внутри класса

## Поле для работы с базой данных

```
private readonly AppDbContext _db;
```

Это приватное поле будет хранить экземпляр контекста базы данных.

Через объект `_db` контроллер сможет:

- получать данные из таблиц
- добавлять записи
- изменять записи
- удалять записи

## Конструктор контроллера

```
// Dependency Injection — EF Core передаёт нам готовый контекст
public TasksController(AppDbContext db) {
    _db = db;
}
```

Здесь используется механизм **Dependency Injection (DI)**.

Когда ASP.NET Core создаёт объект `TasksController`, он автоматически:

1. создаёт экземпляр `AppDbContext`

2. передаёт его в параметр `db`

3. сохраняет его в поле `_db`

После этого контроллер может работать с базой данных.

### Что такое **Dependency Injection (DI)**

**Dependency Injection** — это механизм, при котором **фреймворк сам создаёт необходимые объекты и передаёт их в классы, которым они нужны.**

В нашем случае:

1. В **Program.cs** мы зарегистрировали контекст:

```
builder.Services.AddDbContext<AppDbContext>();
```

2. После этого **ASP.NET Core** знает, **как создавать AppDbContext.**

3. Когда создаётся **TasksController**, фреймворк автоматически передаёт контекст в конструктор.

### Сравнение с обычным созданием объектов

Без DI код выглядел бы примерно так:

```
var db = new AppDbContext(...);
```

То есть разработчик должен:

- сам создавать объект
- сам передавать настройки
- сам следить за временем жизни объекта

С использованием **Dependency Injection**:

- объект создаётся автоматически
- настройки берутся из конфигурации приложения
- ASP.NET Core управляет временем жизни объекта и освобождением ресурсов

**Сравнение с C#:** раньше в консольных приложениях вы писали `new MyClass()` сами.

**DI** делает это за вас — и следит за тем, чтобы объект создавался правильно и вовремя освобождался. Представьте, что вы заказываете пиццу. Без DI — вы сами идёте в магазин, покупаете ингредиенты и готовите. С DI — вы просто говорите «хочу пиццу», и курьер сам привозит готовое. ASP.NET Core — это «курьерская служба»: он знает, как создать нужные объекты, и просто передаёт их вашему классу.

### 5.2. GET — получить все задачи

Добавьте метод в контроллер (наш **класс**):

```
// GET /api/tasks
// GET /api/tasks?completed=true
[HttpGet]
public async Task<ActionResult<IEnumerable<TaskItem>>> GetAll(
    [FromQuery] bool? completed = null,
    [FromQuery] string? priority = null) {
    // Начинаем строить запрос — ещё не выполняем!
    var query = _db.Tasks.AsQueryable();
    // Добавляем условия по мере необходимости
    if (completed.HasValue)
        query = query.Where(t => t.IsCompleted == completed.Value);
    if (!string.IsNullOrWhiteSpace(priority))
        query = query.Where(t => t.Priority == priority);
    // Только здесь запрос уходит в базу данных
    var tasks = await query
        .OrderByDescending(t => t.CreatedAt)
        .ToListAsync();
    return Ok(tasks);
}
```

### Разберём ключевые моменты:

| Элемент                 | Описание                                                        |
|-------------------------|-----------------------------------------------------------------|
| [FromQuery]             | Получает параметры из строки запроса (? completed=true)         |
| async                   | Асинхронный метод — сервер не блокируется во время запроса к БД |
| Task<ActionResult<T>>   |                                                                 |
| AsQueryable()           | Начинаем строить запрос, но ещё не выполняем его                |
| Where(t => ...)         | Добавляем условие — EF Core добавит WHERE в SQL                 |
| await ... ToListAsync() | Только здесь запрос реально уходит в БД и ждём результат        |
| ToListAsync()           | Выполняет запрос к базе данных                                  |
| OrderByDescending       | Сортировка — ORDER BY created_at DESC в SQL                     |

**Почему async/await?** Запрос к базе данных занимает время. Если использовать синхронный вызов, поток сервера будет заблокирован на время ожидания. async/await позволяет серверу обрабатывать другие запросы, пока один ждёт ответа от БД. Это та же концепция, что и async/await в JavaScript!

**AsQueryable() и ленивые запросы:** EF Core не отправляет запрос в БД сразу при вызове Where(). Запрос формируется постепенно и уходит в БД только при

`ToListAsync()`, `FirstOrDefaultAsync()`, `CountAsync()` и т.п. Это называется **deferred execution** (отложенное выполнение).

### 5.3. GET — получить одну задачу

```
// GET /api/tasks/1
[HttpGet("{id}")]
public async Task<ActionResult<TaskItem>> GetById(int id) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });
    return Ok(task);
}
```

`FindAsync(id)` ищет по первичному ключу. Это быстрее, чем `FirstOrDefaultAsync(t => t.Id == id)`, потому что EF Core сначала проверяет кэш (если объект уже был загружен в этом запросе) и только потом идёт в БД.

### 5.4. POST — создать задачу

```
// POST /api/tasks
[HttpPost]
public async Task<ActionResult<TaskItem>> Create([FromBody] CreateTaskDto
dto) {
    if (string.IsNullOrEmpty(dto.Title))
        return BadRequest(new { Message = "Поле Title обязательно для
заполнения" });
    var task = new TaskItem {
        Title = dto.Title.Trim(),
        Description = dto.Description?.Trim() ?? string.Empty,
        Priority = dto.Priority,
        IsCompleted = false,
        CreatedAt = DateTime.UtcNow
    };
    _db.Tasks.Add(task); // Добавляем объект в контекст (ещё не в БД)
    // EF Core выполняет INSERT и обновляет Id
    await _db.SaveChangesAsync();
    return CreatedAtAction(nameof(GetById), new { id = task.Id }, task);
}
```

Класс `CreateTaskDto` мы создадим чуть позже (шаг 5.8). Пока можно написать метод, но до создания DTO сборка будет выдавать ошибку — это нормально.

**Важный момент:** после `_db.Tasks.Add(task)` объект `task` ещё находится в памяти. `SaveChangesAsync()` — это момент, когда EF Core:

1. Составляет SQL `INSERT INTO Tasks (...)`
2. Выполняет его
3. Получает обратно сгенерированный `Id` и записывает его в объект `task`

## 5.5. PUT — обновить задачу

```
// PUT /api/tasks/1
[HttpPut("{id}")]
public async Task<ActionResult<TaskItem>> Update(int id,
[FromBody] UpdateTaskDto dto) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });
    if (string.IsNullOrEmpty(dto.Title))
        return BadRequest(new { Message = "Поле Title не может быть пустым" });
    // EF Core "отслеживает" объект task — любые изменения будут зафиксированы
    task.Title = dto.Title.Trim();
    task.Description = dto.Description?.Trim() ?? string.Empty;
    task.IsCompleted = dto.IsCompleted;
    task.Priority = dto.Priority;
    await _db.SaveChangesAsync(); // Выполняет UPDATE только изменённых полей
    return Ok(task);
}
```

**Change Tracking (отслеживание изменений):** Когда вы загружаете объект из БД через EF Core, он начинает его «отслеживать». Когда вы вызываете `SaveChangesAsync()`, EF Core сравнивает текущее состояние объекта с тем, каким он был при загрузке, и выполняет UPDATE только для изменившихся полей. Вам не нужно писать UPDATE SET ... вручную.

## 5.6. PATCH — переключить статус

```
// PATCH /api/tasks/1/complete
[HttpPatch("{id}/complete")]
public async Task<ActionResult<TaskItem>> ToggleComplete(int id) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });
    task.IsCompleted = !task.IsCompleted;
    await _db.SaveChangesAsync();
    return Ok(task);
}
```

## 5.7. DELETE — удалить задачу

```
// DELETE /api/tasks/1
[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null)
        return NotFound(new { Message = $"Задача с id={id} не найдена" });
    _db.Tasks.Remove(task); // Помечаем объект как "удалить"
    await _db.SaveChangesAsync(); // Выполняет DELETE FROM Tasks WHERE Id = ...
    return NoContent();
}
```



## 5.8. Создание DTO

Для передачи данных через API обычно используют DTO (Data Transfer Object) — специальные классы, предназначенные только для обмена данными между клиентом и сервером.

Создайте файл **Models/TaskDtos.cs** и добавьте следующий код:

```
namespace TaskDb.Models;

public class CreateTaskDto {
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Priority { get; set; } = "Normal";
}

public class UpdateTaskDto {
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public bool IsCompleted { get; set; }
    public string Priority { get; set; } = "Normal";
}
```

### Назначение классов

| Класс                | Используется для                     |
|----------------------|--------------------------------------|
| <b>CreateTaskDto</b> | создания новой задачи (POST)         |
| <b>UpdateTaskDto</b> | обновления существующей задачи (PUT) |

Разделение нужно потому, что при создании задачи некоторые поля задаются сервером автоматически, например:

- Id
- CreatedAt
- IsCompleted

Поэтому клиент отправляет только необходимые данные.

### Почему используют DTO

Использование DTO — распространённая практика при разработке API, потому что:

- модель базы данных не передаётся напрямую клиенту
- можно контролировать, какие поля разрешено изменять
- структура API становится независимой от структуры базы данных.

## Практическое задание

1. Соберите и запустите проект:

**dotnet build**

**dotnet run**

2. Откройте Swagger UI и выполните следующие операции:

- **GET /api/tasks** — убедитесь, что вернулись 3 начальные задачи из seed data
- **POST** — создайте новую задачу
- **GET /api/tasks/{id}** — получите созданную задачу по id
- **PUT** — обновите её
- **PATCH /{id}/complete** — переключите статус
- и — удалите созданную задачу

3. После каждой операции открывайте **taskdb.db** в **SQLite Viewer** и убедитесь, что данные изменились в файле

4. Сделайте скриншот SQLite Viewer с изменившимися данными

5. Сохраните в папку **img** с именем: **step5\_crudLab31-32\_FIO.png**

**Выполните в терминале:**

**cd ..**

**git add .**

**git commit -m "Step 5: TasksController with EF Core CRUD operations"**

**cd TaskDb**

## Шаг 6. LINQ — мощные запросы к базе данных

### 6.1. Что такое LINQ?

**LINQ** (Language Integrated Query) — это встроенный в C# синтаксис для работы с коллекциями данных. EF Core переводит LINQ-запросы в SQL автоматически.

Есть два стиля написания LINQ:

**Стиль 1: Method syntax (методами)** — используем в этой лабораторной

```
var result = _db.Tasks
    .Where(t => t.IsCompleted == false)
    .OrderBy(t => t.CreatedAt)
    .ToList();
```

**Стиль 2: Query syntax (SQL-подобный)** — для справки

```
var result2 = from t in _db.Tasks
               where t.IsCompleted == false
               orderby t.CreatedAt
               select t;
```

В ASP.NET Core разработчики чаще используют **method syntax**. Query syntax похож на SQL и может быть удобнее для тех, кто уже знаком с базами данных. В нашем курсе используем method syntax — он компактнее и чаще встречается в реальных проектах.

## 6.2. Добавляем маршрут поиска с LINQ

Добавьте в **TasksController** новый метод:

```
// GET /api/tasks/search?query=Core&priority=High&completed=false
[HttpGet("search")]
public async Task<ActionResult<IEnumerable<TaskItem>>> Search(
    [FromQuery] string? query = null,
    [FromQuery] string? priority = null,
    [FromQuery] bool? completed = null) {
    var q = _db.Tasks.AsQueryable();
    if (!string.IsNullOrEmpty(query))
        q = q.Where(t =>
            t.Title.Contains(query) ||
            t.Description.Contains(query));
    if (!string.IsNullOrEmpty(priority))
        q = q.Where(t => t.Priority == priority);
    if (completed.HasValue)
        q = q.Where(t => t.IsCompleted == completed.Value);
    var results = await q
        .OrderByDescending(t => t.CreatedAt)
        .ToListAsync();
    return Ok(results);
}
```

Метод позволяет выполнять **поиск задач по нескольким параметрам**.

Пример запроса: `/api/tasks/search?query=core&priority=High`

Разберём ключевые методы LINQ:

| Метод                         | Назначение                                                 |
|-------------------------------|------------------------------------------------------------|
| <b>AsQueryable()</b>          | Начинает формирование запроса к базе данных                |
| <b>Where(...)</b>             | Добавляет условие фильтрации (WHERE в SQL)                 |
| <b>Contains()</b>             | Проверяет, содержит ли строка указанный текст (LIKE в SQL) |
| <b>OrderByDescending(...)</b> | Сортирует результат по убыванию                            |
| <b>ToListAsync()</b>          | Выполняет запрос и возвращает список объектов              |

## Логика метода:

1. Сначала создаётся базовый запрос q.
2. Далее к нему **добавляются фильтры**, если параметры переданы в запросе.
3. После этого выполняется сортировка.
4. Только при вызове ToListAsync() запрос отправляется в базу данных.

Такой подход позволяет **гибко формировать SQL-запрос в зависимости от параметров пользователя**.

## 6.3. Добавляем маршрут статистики с LINQ-агрегацией

Добавьте в **TasksController** новый метод:

```
// GET /api/tasks/stats
[HttpGet("stats")]
public async Task<ActionResult> GetStats() {
    // CountAsync, AnyAsync и другие агрегатные функции
    // сразу уходят в БД — не загружают все записи в память
    var total = await _db.Tasks.CountAsync();
    var completed = await _db.Tasks.CountAsync(t => t.IsCompleted);
    var pending = total - completed;
    // GroupBy — группировка с подсчётом в каждой группе
    var byPriority = await _db.Tasks
        .GroupBy(t => t.Priority)
        .Select(g => new { Priority = g.Key, Count = g.Count() })
        .ToListAsync();
    // Недавно созданные задачи (за последние 7 дней)
    var recentDate = DateTime.UtcNow.AddDays(-7);
    var recentCount = await _db.Tasks
        .CountAsync(t => t.CreatedAt >= recentDate);
    return Ok(new {
        Total = total,
        Completed = completed,
        Pending = pending,
        CompletionPct = total > 0 ? Math.Round((double)completed / total *
100, 1) : 0,
        ByPriority = byPriority,
        CreatedLastWeek = recentCount
    });
}
```

Этот маршрут возвращает **статистику по задачам**.

| Метод                        | Назначение                                       |
|------------------------------|--------------------------------------------------|
| <b>CountAsync()</b>          | Подсчитывает количество записей (COUNT(*) в SQL) |
| <b>CountAsync(condition)</b> | Подсчитывает записи, удовлетворяющие условию     |
| <b>GroupBy(...)</b>          | Группирует данные (GROUP BY в SQL)               |
| <b>Select(...)</b>           | Формирует новый объект из результата запроса     |
| <b>AddDays(-7)</b>           | Получает дату 7 дней назад                       |

**Важно:** методы вроде `CountAsync()` выполняются **не в памяти**, а **непосредственно в базе данных**, поэтому они работают быстрее и не загружают все записи.

В результате API возвращает объект со статистикой:

| Поле            | Значение                               |
|-----------------|----------------------------------------|
| Total           | общее количество задач                 |
| Completed       | выполненные задачи                     |
| Pending         | невыполненные задачи                   |
| CompletionPct   | процент выполненных задач              |
| ByPriority      | количество задач по каждому приоритету |
| CreatedLastWeek | задачи, созданные за последние 7 дней  |

Такой подход часто используется для **создания статистики и аналитики в API**.

#### 6.4. Добавляем пагинацию

Пагинация — это способ разбить большой набор данных на страницы. Клиент получает не все записи сразу, а только небольшую часть (страницу).

Добавьте в контроллер следующий метод:

```
// GET /api/tasks/paged?page=1&pageSize=2
[HttpGet("paged")]
public async Task<ActionResult> GetPaged(
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 5) {
    if (page < 1) page = 1;
    if (pageSize < 1) pageSize = 5;
    if (pageSize > 50) pageSize = 50; // защита от слишком больших запросов
    var totalCount = await _db.Tasks.CountAsync();
    var totalPages = (int)Math.Ceiling((double)totalCount / pageSize);
    var tasks = await _db.Tasks
        .OrderByDescending(t => t.CreatedAt)
        .Skip((page - 1) * pageSize) // пропустить записи предыдущих страниц
        .Take(pageSize) // взять только нужное количество
        .ToListAsync();
    return Ok(new {
        Page = page,
        PageSize = pageSize,
        TotalCount = totalCount,
        TotalPages = totalPages,
        HasPrev = page > 1,
        HasNext = page < totalPages,
        Items = tasks
    });
}
```

Пример запроса: `/api/tasks/paged?page=2&pageSize=5`

Ограничение в 50 записей на страницу — это защита сервера. Если клиент запросит 100 000 записей за раз, сервер может упасть от нагрузки. Такие ограничения — стандартная практика в реальных API.

Это означает: **получить вторую страницу по 5 задач на странице.**

### Разбор ключевых методов

| Метод          | Назначение                                   |
|----------------|----------------------------------------------|
| [FromQuery]    | Получает параметры из строки запроса         |
| CountAsync()   | Считает общее количество записей             |
| Skip(n)        | Пропускает указанное количество строк        |
| Take(n)        | Возвращает только указанное количество строк |
| Math.Ceiling() | Округляет число страниц вверх                |

### Логика метода:

1. Проверяются параметры page и pageSize.
2. Получается **общее количество задач** (totalCount).
3. Вычисляется **общее количество страниц** (totalPages).
4. Через Skip() и Take() выбираются записи нужной страницы.
5. Возвращается объект с данными страницы и навигацией.

### Как работает Skip() и Take()

*.Skip((page - 1) \* pageSize)*

*.Take(pageSize)*

Пример:

| Page | PageSize | Пропускаем | Берём |
|------|----------|------------|-------|
| 1    | 5        | 0          | 5     |
| 2    | 5        | 5          | 5     |
| 3    | 5        | 10         | 5     |

### Почему нужна пагинация

Если в таблице **сотни тысяч или миллионы записей**, загружать их все сразу нельзя:

- большой объём данных
- высокая нагрузка на сервер
- медленные ответы API

Методы Skip() и Take() преобразуются EF Core в SQL: *LIMIT pageSize OFFSET ...*

Поэтому база данных возвращает **только нужную страницу записей**, а не всю таблицу.

## 6.5. Сравнение LINQ и SQL

Посмотрите, как LINQ-методы соответствуют SQL:

| LINQ                                 | SQL                                        |
|--------------------------------------|--------------------------------------------|
| .Where(t => t.IsCompleted == false)  | WHERE is_completed = 0                     |
| .OrderBy(t => t.CreatedAt)           | ORDER BY created_at ASC                    |
| .OrderByDescending(t => t.CreatedAt) | ORDER BY created_at DESC                   |
| .Take(10)                            | LIMIT 10                                   |
| .Skip(20).Take(10)                   | OFFSET 20 LIMIT 10                         |
| .Count()                             | SELECT COUNT(*)                            |
| .Any(t => t.Priority == "High")      | SELECT EXISTS(... WHERE priority = 'High') |
| .Max(t => t.Id)                      | SELECT MAX(id)                             |
| .GroupBy(t => t.Priority)            | GROUP BY priority                          |
| .Select(t => t.Title)                | SELECT title                               |

## Практическое задание

1. Создайте в БД минимум **6 задач** через POST в Swagger (с разными приоритетами и статусами), чтобы было удобнее тестировать:

- 2 задачи с приоритетом High, 1 выполненная
- 2 задачи с приоритетом Normal
- 2 задачи с приоритетом Low, 1 выполненная

2. Проверьте новые маршруты в Swagger или REST Client:

### ### Поиск по тексту

**GET** <http://localhost:5000/api/tasks/search?query=SQL>

### ### Поиск с несколькими фильтрами

**GET** <http://localhost:5000/api/tasks/search?priority=High&completed=false>

### ### Статистика

**GET** <http://localhost:5000/api/tasks/stats>

### ### Пагинация — страница 1

**GET** <http://localhost:5000/api/tasks/paged?page=1&pageSize=3>



### ### Пагинация — страница 2

GET `http://localhost:5000/api/tasks/paged?page=2&pageSize=3`

3. Сделайте скриншот результата маршрута `/stats`

4. Сохраните в папку `img` с именем: **step6\_linqLab32\_FIO.png**

Выполните в терминале:

**cd ..**

**git add .**

**git commit -m "Step 6: LINQ queries - search, stats, pagination"**

**git push**

**cd TaskDb**

## Шаг 7. Изменение схемы БД — новая миграция

### 7.1. Зачем это нужно?

Одно из главных преимуществ **Code First** — вы можете изменить класс `C#` и применить изменение к БД через миграцию. Не нужно вручную писать **ALTER TABLE**.

### 7.2. Добавление нового поля в модель

Откройте **Models/TaskItem.cs** и добавьте новое поле **DueDate** (срок выполнения) внутри класса **TaskItem**:

```
public DateTime? DueDate { get; set; }
```

### 7.3. Создание и применение новой миграции

Выполните в терминале:

**dotnet ef migrations add AddDueDateToTask**

**dotnet ef database update**

Откройте файл новой миграции и изучите его — EF Core сгенерировал только **AddColumn**, не трогая существующие данные:



### 7.4. Обновление DTO

Добавьте поле **DueDate** в оба DTO в файле **Models/TaskDtos.cs**:



```

public class CreateTaskDto {
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Priority { get; set; } = "Normal";
    + public DateTime? DueDate { get; set; } // новое поле
}

public class UpdateTaskDto {
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public bool IsCompleted { get; set; }
    public string Priority { get; set; } = "Normal";
    + public DateTime? DueDate { get; set; } // новое поле
}

```

## 7.5. Обновление методов контроллера

Откройте файл **TasksController.cs**

В методе **Create** добавьте заполнение нового поля:

```

    IsCompleted = false,
    CreatedAt = DateTime.UtcNow,
    + DueDate = dto.DueDate // новое поле

```

В методе **Update** добавьте обновление:

```

    task.Priority = dto.Priority;
    + task.DueDate = dto.DueDate; // новое поле
    await _db.SaveChangesAsync();

```

Добавьте также маршрут для получения просроченных задач:

```

// GET /api/tasks/overdue
[HttpGet("overdue")]
public async Task<ActionResult<IEnumerable<TaskItem>>> GetOverdue() {
    var now = DateTime.UtcNow;
    var overdue = await _db.Tasks
        .Where(t => t.DueDate != null
            && t.DueDate < now
            && !t.IsCompleted)
        .OrderBy(t => t.DueDate)
        .ToListAsync();
    return Ok(overdue);
}

```

## Практическое задание

1. Соберите и запустите проект:

**dotnet build**

**dotnet run**

2. В **Swagger** проверьте, что поле **dueDate** появилось в схеме **POST** и **PUT** запросов

3. Создайте задачу с датой в прошлом (например, "dueDate": "2025-01-01T00:00:00Z")

4. Проверьте маршрут **GET /api/tasks/overdue** — просроченная задача должна появиться

5. Откройте **taskdb.db** в **SQLite Viewer** и убедитесь, что колонка **DueDate** появилась в таблице

6. Выполните **dotnet ef migrations list** — должны быть обе миграции

7. Сделайте скриншот терминала и **SQLite Viewer**

8. Сохраните в папку **img** с именем: **step7\_migrationLab32\_FIO.png**

Выполните в терминале:

**cd ..**

**git add .**

**git commit -m "Step 7: AddDueDateToTask migration, overdue endpoint"**

**cd TaskDb**

## Шаг 8. Логирование SQL-запросов

### 8.1. Включение логирования

EF Core умеет выводить в консоль сгенерированный SQL. Это очень полезно для понимания того, какие запросы реально уходят в БД.

Добавьте настройку логирования в **appsettings.Development.json**:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  },
  ...
}
```

## 8.2. Наблюдение за SQL в терминале

1. Запустите сервер: **dotnet run**
2. Выполните GET **/api/tasks** в Swagger
3. Посмотрите в терминал — вы увидите реальный SQL-запрос:

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (8ms) [Parameters=[@completed_Value='?' (DbType = Boolean), @priority='?' (Size = 1
)], CommandType='Text', CommandTimeout='30']
      SELECT "t"."Id", "t"."CreatedAt", "t"."Description", "t"."DueDate", "t"."IsCompleted", "t"."Priority",
      "t"."Title"
      FROM "Tasks" AS "t"
      WHERE "t"."IsCompleted" = @completed_Value AND "t"."Priority" = @priority
      ORDER BY "t"."CreatedAt" DESC
```

## 8.3. Практический эксперимент

Выполните разные запросы и наблюдайте за SQL:

| Запрос к API                           | Какой SQL вы увидите      |
|----------------------------------------|---------------------------|
| GET /api/tasks                         | SELECT * с ORDER BY       |
| GET /api/tasks?completed=false         | SELECT * с WHERE          |
| GET /api/tasks/search?query=SQL        | SELECT * с WHERE LIKE     |
| GET /api/tasks/stats                   | Несколько SELECT COUNT(*) |
| GET /api/tasks/paged?page=2&pageSize=3 | SELECT с LIMIT и OFFSET   |

### Практическое задание

1. Запустите сервер и выполните несколько разных запросов
2. Сделайте скриншот терминала с SQL-запросами (видны 2–3 разных запроса)
3. Сохраните в папку img с именем: **step8\_sqlLogsLab32\_FIO.png**

Выполните в терминале:

**cd ..**

**git add .**

**git commit -m "Step 8: SQL logging enabled, observed generated queries"**

**cd TaskDb**

**git push**

## Самостоятельное задание

### Задание 1. Маршрут массового обновления

Реализуйте маршрут **PATCH** `/api/tasks/complete-all`, который помечает **все невыполненные** задачи как выполненные. Метод должен возвращать количество обновлённых задач.

**Подсказка:** можно использовать `ExecuteUpdateAsync` (EF Core 7+) или загрузить все задачи и обновить в цикле.

```
// Вариант 1 – через цикл (проще для понимания):
var tasks = await _db.Tasks.Where(t => !t.IsCompleted).ToListAsync();
foreach (var task in tasks) {
    task.IsCompleted = true;
}
await _db.SaveChangesAsync();
return Ok(new { Updated = tasks.Count });

// Вариант 2 – через ExecuteUpdateAsync (эффективнее, один SQL-запрос):
var count = await _db.Tasks
    .Where(t => !t.IsCompleted)
    .ExecuteUpdateAsync(s => s.SetProperty(t => t.IsCompleted, true));
return Ok(new { Updated = count });
```

### Задание 2. Маршрут удаления выполненных задач

Реализуйте маршрут **DELETE** `/api/tasks/completed`, который удаляет **все выполненные** задачи. Метод должен возвращать количество удалённых задач.

**Подсказка:** `ExecuteDeleteAsync` или `RemoveRange`.

### Задание 3. README.md

Создайте подробный файл `README.md` со следующим содержанием:

1. Заголовок и информация о студенте
2. Краткое описание работы
3. Полезные команды dotnet ef
4. Структура проекта (дерево файлов)
5. Список всех реализованных маршрутов
6. Таблица применённых миграций
7. Сравнительная таблица LINQ vs SQL

## 8. Главные выводы (минимум 5)

**Итоговая сравнительная таблица (обязательно включить в README.md):**

| Концепция          | Хранение в памяти        | EF Core + SQLite                           |
|--------------------|--------------------------|--------------------------------------------|
| Хранение данных    | static List<T> в RAM     | Файл .db на диске                          |
| После перезапуска  | Данные пропадают         | Данные сохраняются                         |
| Поиск по условию   | LINQ to Objects          | LINQ to Entities → SQL                     |
| Создание структуры | Не нужно                 | Миграции (dotnet ef)                       |
| Начальные данные   | Хардкод в коде           | HasData() в миграции                       |
| Получение данных   | list.FirstOrDefault(...) | await db.Table.FindAsync(id)               |
| Добавление         | list.Add(item)           | db.Table.Add(item) + SaveChangesAsync()    |
| Удаление           | list.Remove(item)        | db.Table.Remove(item) + SaveChangesAsync() |
| Масштабируемость   | Ограничена RAM           | Гигабайты данных                           |
| Транзакции         | Нет                      | Встроены в EF Core                         |

**Главные выводы:**

1. **EF Core** — это переводчик между **C#** и **SQL**. Вы пишете **LINQ**, он пишет **SQL**.
2. **Миграции** — это система контроля версий для структуры БД. Так же, как Git для кода.
3. **Code First** удобнее, чем писать SQL вручную: изменил класс → создал миграцию → база обновлена.
4. **SaveChangesAsync()** — ключевой момент. До него все изменения живут только в памяти.
5. **async/await** при работе с БД — это не опционально, а стандарт. Блокировать поток на время ожидания БД — плохая практика.

**Выполните в терминале:**

**cd ..**

**git add .**

**git commit -m "Added complete README for Lab31-32"**

**cd TaskDb**

**git push**

## Итоговая таблица: что изучили в лабораторных

| Команда / концепция                           | Описание                                              |
|-----------------------------------------------|-------------------------------------------------------|
| <b>dotnet add package</b>                     | Установить NuGet-пакет (аналог npm install)           |
| <b>dotnet tool install --global dotnet-ef</b> | Установить инструмент для работы с миграциями         |
| <b>dotnet ef migrations add Имя</b>           | Создать новую миграцию по изменениям в моделях        |
| <b>dotnet ef database update</b>              | Применить все непримененные миграции к БД             |
| <b>dotnet ef migrations list</b>              | Посмотреть список миграций и их статус                |
| <b>AppDbContext</b>                           | Главный класс EF Core — представляет подключение к БД |
| <b>DbSet&lt;T&gt;</b>                         | Коллекция-таблица в коде C#                           |
| <b>HasData(...)</b>                           | Начальные данные (seed), добавляемые при создании БД  |
| <b>AsQueryable()</b>                          | Начать строить отложенный запрос к БД                 |
| <b>ToListAsync()</b>                          | Выполнить запрос и получить список                    |
| <b>FindAsync(id)</b>                          | Найти запись по первичному ключу                      |
| <b>SaveChangesAsync()</b>                     | Зафиксировать все изменения в БД                      |
| <b>Where(t =&gt; ...)</b>                     | Фильтрация — WHERE в SQL                              |
| <b>OrderBy / OrderByDescending</b>            | Сортировка — ORDER BY в SQL                           |
| <b>Skip(n).Take(m)</b>                        | Пагинация — OFFSET n LIMIT m в SQL                    |
| <b>CountAsync(...)</b>                        | Подсчёт записей — SELECT COUNT(*) в SQL               |
| <b>GroupBy(...)</b>                           | Группировка — GROUP BY в SQL                          |
| <b>[Required], [MaxLength]</b>                | Атрибуты валидации + ограничения в БД                 |
| <b>DateTime? (nullable)</b>                   | Необязательное поле — NULL допустим в БД              |
| <b>Code First</b>                             | Подход: сначала код C#, потом БД по нему              |